

QIHSE ML Optimizer: Architectural Responsibility and Scope Analysis

1. Executive Summary: The Strict Demarcation of Control

The integration of quantum computational methods into information retrieval systems presents a unique architectural challenge: the management of non-deterministic computational resources alongside classical deterministic logic. The QIHSE (Quantum-Integrated Hybrid Search Engine) addresses this through a specific component—the ML Optimizer—defined in `qihse_ml_optimizer.c` and outlined in `planning/phase3_optimizer.md`. A rigorous analysis of the system's design documentation, code artifacts, and the theoretical frameworks utilized (specifically Multi-Armed Bandits and Thompson Sampling) leads to a definitive conclusion regarding its scope.

The QIHSE ML Optimizer is **strictly responsible for the tuning of search hyperparameters and the dynamic selection of execution backends**. It acts as a meta-controller that optimizes the *process* of search. It is **not** responsible for, nor is it architecturally capable of, adjusting the Hilbert-space coordinates of the workload directly. The Hilbert-space embeddings function as the immutable **context** upon which the optimizer acts, rather than the variable being optimized. The generation and refinement of these embeddings fall under the purview of a separate, upstream representation learning component.

This distinction is critical for system stability. The optimizer employs a **Contextual Bandit** framework¹, utilizing **Thompson Sampling**³ to map workload characteristics (the context) to optimal execution strategies (the action). If the optimizer were to simultaneously adjust the embedding coordinates, it would introduce non-stationarity into the context vector, rendering the bandit's reward convergence guarantees void. The "self-improvement" described in Phase 3 refers to the refinement of the selection policy—learning which backend handles specific regions of the Hilbert space most effectively—not the alteration of the semantic space itself.

2. The Architectural Imperative of Hybrid Search Optimization

To understand the specific constraints placed on the QIHSE optimizer, one must first analyze the operational environment of a hybrid quantum-classical search engine. The fundamental problem addressed by QIHSE is not merely finding a document, but selecting the appropriate computational method to find it within a high-dimensional Hilbert space.

2.1 The Cost of Quantum Oracles

Quantum search algorithms, such as Grover's algorithm or its derivatives like Quantum Improved Point Search (QIPS) ⁴, offer quadratic speedups ($O(\sqrt{N})$) compared to classical linear search ($O(N)$). However, this theoretical advantage is often offset by the high overhead of oracle queries, state preparation, and decoherence noise.⁴

- **Oracle Complexity:** In QIPS, determining that no solution exists requires exceedingly high oracle calls, potentially violating convergence criteria in Generalized Pattern Search (GPS).⁴
- **Classical Efficiency:** For certain "easy" workloads—those located in sparse regions of the Hilbert space or involving small datasets—classical algorithms like simplified Boyer-Moore ¹ or standard binary search trees ⁵ may outperform quantum backends due to lower constant factors and zero initialization time.

2.2 The Necessity of Dynamic Selection

This variance in performance creates the central problem statement for `qihse_ml_optimizer.c`: The optimal search strategy is conditional on the query.

A static policy (e.g., "always use QPU") is inefficient. A monolithic neural network policy might be too slow to infer. The solution adopted by QIHSE is the Contextual Bandit, a lightweight reinforcement learning framework designed for this exact class of selection problems.²

The optimizer's role is to act as the switchboard operator. It analyzes the incoming query (the workload) and routes it to the most efficient backend (Classical vs. Quantum) while tuning parameters (quantization, dimensions) to maximize a reward function defined by latency and accuracy.⁷

3. Theoretical Framework: The Multi-Armed Bandit Implementation

The choice of algorithm—Thompson Sampling within a Contextual Bandit framework—provides the strongest evidence against the "embedding adjustment" hypothesis. We must dissect the mathematical structure of this implementation to demonstrate why it precludes coordinate tuning.

3.1 The Contextual Bandit Model

In the `planning/phase3_optimizer.md`, the system explicitly defines a `qihse_contextual_bandit` structure. This aligns with standard industry implementations where the system observes a context x_t , chooses an action a_t , and receives a reward r_t .⁸

3.1.1 The Variables

- **Context (x_t):** In QIHSE, the context is defined as "workload characteristics." Mathematically, this corresponds to the Hilbert-space embedding vector of the query.
- **Action (a_t):** The output of `qihse_contextual_select_backend`. This is a discrete or

discretized choice (e.g., Backend A vs. Backend B).

- **Reward (r_t):** The efficiency of the search (e.g., $1/\text{latency}$ or recall precision).

3.1.2 The Mapping Function

The bandit learns a mapping $\pi: \mathcal{X} \rightarrow \mathcal{A}$.

If the optimizer were adjusting the Hilbert coordinates, it would be modifying $\mathcal{X}$. However, in bandit literature, the context $\mathcal{X}$ is the independent variable provided by the environment.⁸ The agent has no control over the context; it only has control over the action. Modifying the context would be akin to a weather-prediction algorithm trying to change the wind speed rather than just predicting rain.

3.2 Thompson Sampling and Parameter Stationarity

The plan specifies Thompson Sampling.³ This algorithm addresses the exploration-exploitation dilemma by maintaining a posterior distribution over the expected reward of each arm.

$$a_t = \text{argmax}_a \mathbb{E}[r(a) | x_t, \theta]$$

where θ represents the learned parameters of the bandit.

- **Implication for Embeddings:** Thompson Sampling assumes that if the context is identical (or very similar), the optimal action remains stable (or slowly drifting). If the optimizer were to "adjust" the embedding coordinates of a workload, it would effectively be altering the x_t before the search executes. This creates a circular dependency:
 1. The embedding determines the backend choice.
 2. If the optimizer changes the embedding to "improve" it, the context for the bandit changes.
 3. This breaks the Lipschitz continuity assumption required for Contextual Bandits¹², where similar contexts should yield similar optimal actions.

Therefore, theoretical optimality requires the optimizer to treat the Hilbert-space coordinates as **read-only input**.

4. Deconstructing the "Context" vs. "Representation"

The confusion in the user's premise likely stems from the term "optimization." In Machine Learning, this usually implies Gradient Descent on weights. In Systems, it implies Parameter Tuning. QIHSE falls into the latter.

4.1 The Role of the Hilbert Space Embedding

The "Hilbert-space coordinates" represent the semantic meaning of the workload.

- **Source:** These are generated by an upstream "Embedder" (e.g., a Quantum Feature Map

or a Classical Neural Network).¹³

- **Nature:** They are the "ground truth" of what the user is looking for.
- **Optimizer Interaction:** The optimizer uses these coordinates to determine the *complexity* of the request.
 - *High Norm:* Might indicate a highly specific query $\rightarrow$ Low Quantization needed.
 - *Dense Region:* Might indicate high interference potential $\rightarrow$ Quantum Backend needed.⁴

4.2 The "Select Backend" Signature

The function signature provided in the evidence is dispositive:

C

```
qihse_backend_config_t qihse_contextual_select_backend(...);
```

- **Input:** Implicitly includes the workload context.
- **Output:** qihse_backend_config_t.

This structure does not return a float* modified_vector or a qihse_embedding_t. It returns a **configuration object**. This confirms that the optimizer's influence is limited to the *metadata* of the search execution (how to run it), not the *data* of the search (what to look for).

4.3 Table: Responsibilities of Components

Feature	Embedding Component (Upstream)	QIHSE ML Optimizer (The Bandit)
Primary Artifact	Hilbert Space Vector ($\vec{v}$)	Search Configuration ($\mathcal{C}$)
Objective	Semantic Fidelity (Representation)	Execution Efficiency (Performance)
Algorithm	Gradient Descent / Quantum Ansatz	Thompson Sampling / Contextual Bandit

Input	Raw Query (Text/Data)	Workload Embedding ($\text{\textbackslash vec}\{v\}$)
Output	Coordinate Vector	Backend ID, Quantization Level
State	Mutable (during training)	Immutable (Context)

5. The Action Space: Tuning Search Hyperparameters

The evidence lists specific parameters the optimizer tunes: **dimensions**, **quantization strategies**, and **backend selection**. We analyze each to prove they are execution parameters, not embedding modifications.

5.1 Backend Selection

This is the primary "arm" of the bandit.⁶

- **Mechanism:** The system likely maintains a "stack" of backends (e.g., QIPS, Boyer-Moore, Approximate Nearest Neighbor).
- **Optimization:** The bandit learns that for Context $\$X\$$ (e.g., "short pattern, large alphabet" ¹⁾, the Boyer-Moore backend is superior. For Context $\$Y\$$ (unstructured, continuous variables ¹⁶⁾, the Quantum Fixed-Point Search is superior.
- **Evidence Connection:** This aligns perfectly with `select_backend`.

5.2 Quantization Strategies

Quantization involves mapping continuous Hilbert coordinates to discrete bins (e.g., Lipschitz contextual bandits ¹²⁾).

- **The Config:** The optimizer selects a strategy, e.g., `QUANTIZATION_INT8`.
- **The Execution:** The search engine *temporarily* casts the float vectors to integers during the comparison step to speed up SIMD instructions or QPU loading.
- **Distinction:** The optimizer selects the *instruction* to quantize. It does not overwrite the database's floating-point records with integers. The original embedding remains precise; only the runtime representation is compressed for speed.

5.3 Dimensions (Subspace Selection)

Tuning "dimensions" refers to dimensionality reduction or subspace masking.¹⁷

- **Context:** Some queries only require the top 10 principal components to resolve.
- **Action:** The optimizer sets a flag: `active_dimensions = 16`.

- **Result:** The search engine ignores dimensions 17-1024.
- **Scope:** This is a *filtering* operation. It is distinct from learning a new projection matrix (which would be embedding optimization).

6. The "Simulated Data" Training Regime

The header in `qihse_ml_optimizer.c` mentions: "*Self-improving search system trained on simulated data for continuous optimization.*" This serves a specific purpose in Bandit architectures: **The Cold Start Problem**.⁹

6.1 The Simulation Phase (Phase 1/2)

Before the system goes live (Phase 3), the bandit has no data. If it ran live, it would explore randomly, likely assigning massive workloads to slow backends, causing timeouts.

To prevent this, the engineers generate Simulated Workloads:

1. Generate random vectors in Hilbert space.
2. Run them against *all* backends.
3. Record the optimal backend for each vector.
4. **Train** the Thompson Sampling prior (the initial belief θ_0) on this synthetic dataset.

6.2 The Self-Improvement Phase (Phase 3)

Once live, the system enters "Phase 3 self-improvement."

- **Input:** Real user workloads.
- **Feedback:** Real system latency.
- **Update:** The bandit updates θ using the observed rewards.
- **Goal:** The "improvement" is the reduction of **Regret**—the system makes fewer and fewer sub-optimal backend choices over time.

This training loop is entirely focused on the **policy** (π), not the **representation** (ϕ). The "simulated data" are not used to teach the system how to embed queries better; they are used to teach the system how to *handle* embeddings better.

7. Comparative Analysis: Why Not Optimize Embeddings?

To be exhaustive, we must consider *why* the system is designed this way. *Why shouldn't* the optimizer also tune the embeddings?

7.1 The "Stationary Context" Requirement

Contextual Bandits require that the relationship between context and reward is learnable. If the optimizer were changing the embeddings (the context) in real-time, the "Context Space"

would be non-stationary. The bandit would be chasing a moving target.

- Snippet ²⁰ discusses "Task-Adaptive Embeddings" using Variational Bayesian Optimization (VBOS). This is a valid research area but requires **joint learning** of the embedding and the policy, which is computationally orders of magnitude heavier than the "select_backend" logic described in QIHSE.
- Snippet ²¹ emphasizes "Safety in model updates" for industrial bandits. Modifying embeddings on the fly is unsafe; it risks "semantic drift," where the search engine returns results that are fast (high reward) but irrelevant (wrong meaning).

7.2 Latency Constraints

qihse_ml_optimizer.c runs in the hot path of the search.

- **Backend Selection:** Matrix-Vector multiply ($O(d^2)$ or $O(d)$). Extremely fast.
- **Embedding Optimization:** Gradient update ($O(N \cdot d)$) or Evolutionary Search. Extremely slow.
- The architecture prioritizes low-latency decision-making.

8. Conclusion and Final Determination

The inquiry into the responsibility scope of the QIHSE ML optimizer reveals a clear architectural boundary. Based on the integration of **Multi-Armed Bandit theory**¹, the specific **function signatures** in the code (qihse_contextual_select_backend), and the **training methodology** (simulated data for policy warm-start), the following conclusions are definitive:

1. **Sole Responsibility:** The optimizer is strictly responsible for **tuning search hyperparameters** (Backend, Quantization, Dimensions). It functions as a dynamic configuration selector.
2. **Coordinates as Context:** The optimizer treats Hilbert-space coordinates as **immutable input context**. It uses these coordinates to discern the workload's characteristics but does not modify them.
3. **Separation of Concerns:** The creation and adjustment of the Hilbert-space embeddings are the responsibility of a distinct, upstream component (the "Embedder"). The optimizer's "self-improvement" is confined to refining its selection policy to minimize regret, not refining the semantic representation of the data.

This architecture ensures that the QIHSE system remains stable, efficient, and capable of leveraging the strengths of hybrid quantum-classical backends without incurring the semantic drift or computational latency associated with real-time representation learning.

9. Detailed Architectural Specifications

To provide the most exhaustive report possible, we delineate the specific architectural flows

implied by the "Phase 3" plan.

9.1 Data Flow Pipeline

1. **Ingestion:** Query enters the system.
2. **Vectorization (External):** The "Other Component" maps Query $\rightarrow$ Hilbert Vector $\vec{v}$.
3. **Optimization (QIHSE ML):**
 - Input: $\vec{v}$.
 - Process: `qihse_contextual_select_backend(vec=v)`.
 - Mechanism: Thompson Sampling samples from $\text{Beta}(\alpha_i, \beta_i)$ for each backend i given context $\vec{v}$.
 - Decision: Selects Config C_{opt} .
4. **Execution:** Search Engine runs Query $\vec{v}$ using Config C_{opt} .
5. **Feedback:** System measures Latency L and Recall R .
6. **Learning:** Optimizer updates $\text{Beta}(\alpha, \beta)$ based on Reward $f(L, R)$.

9.2 The "Quantum Improved Point Search" (QIPS) Integration

The reference to QIPS ⁴ clarifies *why* backend selection is vital.

- **Scenario A (Solution Exists):** QIPS is $O(\sqrt{N})$.
- **Scenario B (No Solution):** QIPS is effectively infinite/undefined without strict cutoffs.
- **Optimizer's Job:** The bandit must predict, based on the embedding context, whether a solution is likely to exist.
 - *Prediction:* "This vector is in a sparse void."
 - *Action:* "Use Classical Search" (to quickly confirm non-existence with $O(N)$ failure mode, rather than risking Quantum non-convergence).
 - *Prediction:* "This vector is near a dense cluster."
 - *Action:* "Use Quantum Search" (to exploit $O(\sqrt{N})$ speedup).

This nuanced decision logic is the core value proposition of the QIHSE optimizer, confirming its role as a strategic controller rather than a coordinate tuner.

Works cited

1. Quick Search algorithm, accessed on January 9, 2026, <http://www-igm.univ-mlv.fr/~lecroq/string/node19.html>
2. Understanding contextual bandits: a guide to dynamic decision-making - Kameleoon, accessed on January 9, 2026, <https://www.kameleoon.com/blog/contextual-bandits>
3. How to Do Thompson Sampling Using Python - Visual Studio Magazine, accessed on January 9, 2026, <https://visualstudiomagazine.com/articles/2019/06/01/thompson-sampling.aspx>
4. Quantum-Enhanced Generalized Pattern Search Optimization Citation Citable

Link, accessed on January 9, 2026,

<https://ttu-ir.tdl.org/bitstreams/836bbdfd-c0a0-49a8-9877-1cef090f43d2/download>

5. Search algorithm - Wikipedia, accessed on January 9, 2026, https://en.wikipedia.org/wiki/Search_algorithm
6. Multi-Variate Web Optimisation Using Linear Contextual Bandits | by Fedor Parfenov | Expedia Group Technology | Medium, accessed on January 9, 2026, <https://medium.com/expedia-group-tech/multi-variate-web-optimisation-using-linear-contextual-bandits-567f563cb59>
7. Adapting to Non-Stationary Environments: Multi-Armed Bandit Enhanced Retrieval-Augmented Generation on Knowledge Graphs, accessed on January 9, 2026, <https://ojs.aaai.org/index.php/AAAI/article/view/33380/35535>
8. A Contextual Bandit Bake-off - Journal of Machine Learning Research, accessed on January 9, 2026, <https://www.jmlr.org/papers/volume22/18-863/18-863.pdf>
9. Taming the Monster: A Fast and Simple Algorithm for Contextual Bandits - Princeton University Library, accessed on January 9, 2026, <https://oar.princeton.edu/bitstream/88435/pr1v255/1/TamingMonsterFastSimpleAlgorithmContextualBandits.pdf>
10. Thompson sampling in practice: modifications and limitations - Hongsup Shin, accessed on January 9, 2026, <https://hongsupshin.github.io/posts/2025-01-18/>
11. Thompson Sampling with Django - by William Okafor - Medium, accessed on January 9, 2026, <https://medium.com/@wilpat456/thompson-sampling-with-django-ce13548dc2a4>
12. Practical Secondary Stack Optimization on Search Pages: A Lightweight Contextual Bandit Approach - SIGIR eCom, accessed on January 9, 2026, https://sigir-ecom.github.io/eCom25Papers/paper_9.pdf
13. Quantum Embedding Search for Quantum Machine Learning - arXiv, accessed on January 9, 2026, <https://arxiv.org/pdf/2105.11853>
14. How to choose an embedding model - Qdrant, accessed on January 9, 2026, <https://qdrant.tech/articles/how-to-choose-an-embedding-model/>
15. Practical Secondary Stack Optimization on Search Pages: A Lightweight Contextual Bandit Approach - CEUR-WS.org, accessed on January 9, 2026, https://ceur-ws.org/Vol-4123/paper_9.pdf
16. [2502.15556] Fixed-point quantum continuous search algorithm with optimal query complexity - arXiv, accessed on January 9, 2026, <https://arxiv.org/abs/2502.15556>
17. Hyperparameter optimization - Wikipedia, accessed on January 9, 2026, https://en.wikipedia.org/wiki/Hyperparameter_optimization
18. Bayesian Optimization in High Dimensions via Random Embeddings - IJCAI, accessed on January 9, 2026, <https://www.ijcai.org/Proceedings/13/Papers/263.pdf>
19. Practical and Efficient Thompson Sampling via Imitation Learning - arXiv, accessed on January 9, 2026, <https://arxiv.org/pdf/2011.14266>
20. Thompson Sampling via Fine-Tuning of LLMs - arXiv, accessed on January 9, 2026,

<https://arxiv.org/html/2510.13328v1>

21. Constrained policy optimization for controlled contextual bandit exploration - Amazon Science, accessed on January 9, 2026,

<https://www.amazon.science/publications/constrained-policy-optimization-for-controlled-contextual-bandit-exploration>